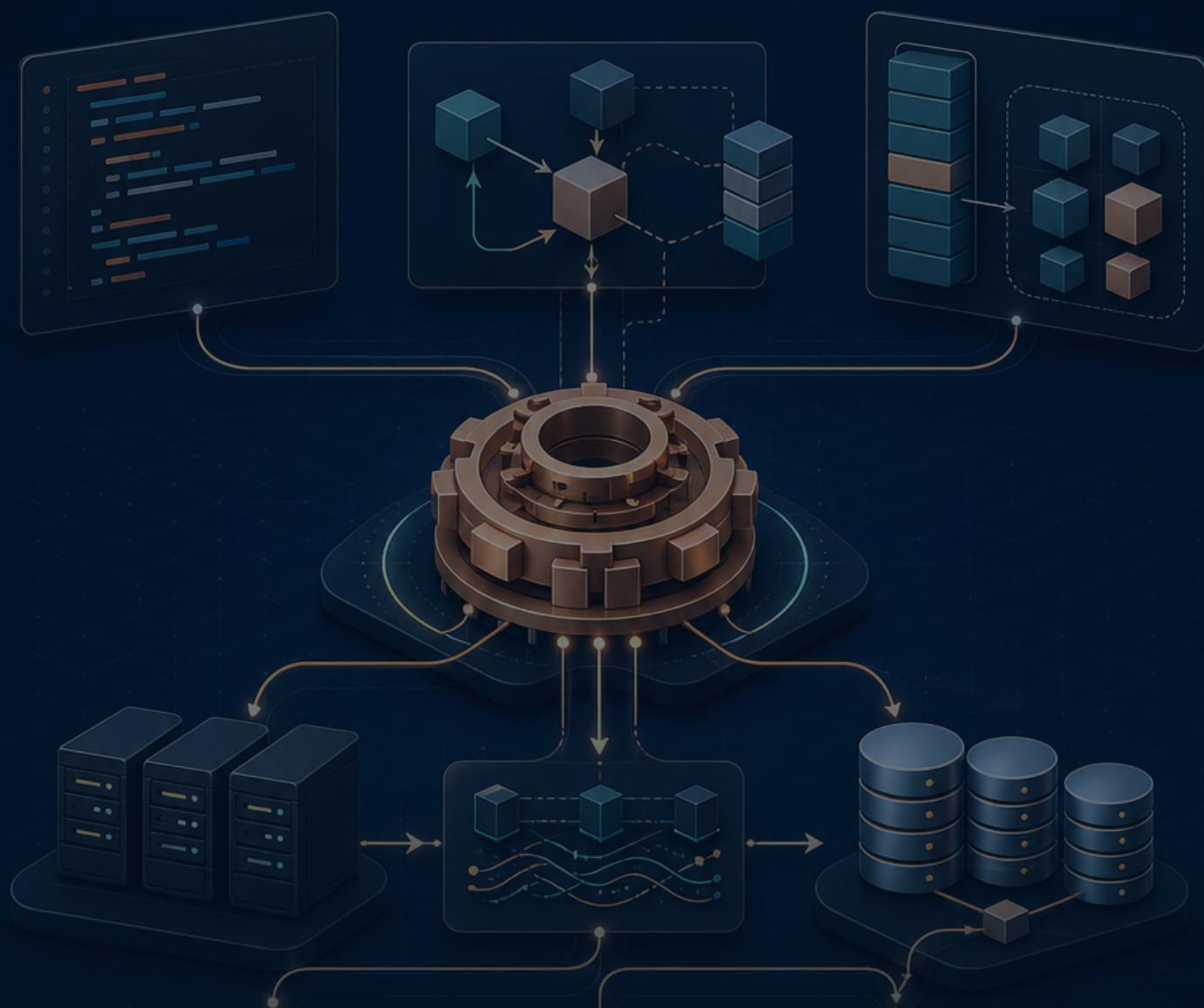




VOLUME 06 / RUST BACKEND MASTERY

Web基礎とTopcoat 0.5.0完全ガイド

HTTPの一周からTopcoat固有API、SSR、assets、runtime、sessionまでを公式0.5.0に固定して学ぶ

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

Topcoatの便利さだけでなく、抽象化の下にあるHTTP、未信頼入力、公開endpoint、asset配布を見える形にします。

到達目標

- ✓ HTTP要求と応答を読む
- ✓ Topcoat 0.5.0でpage・route・layoutを書く
- ✓ Cx・parameter・form・contextを型安全に扱える
- ✓ assets・runtime・sessionの運用上の境界を説明できる

学習地図

01 HTTP要求・応答・状態コード

HTTPはmethod、path、headers、bodyの要求と、status、headers、bodyの応答です。GETは安全、PUT/DELETEは冪等など、意味を守るとcache、retry、proxyが正しく働きます。

02 Topcoat 0.5.0最小アプリ

Cargo binaryへtopcoat 0.5.0とTokioを追加し、pageとcomponentをdiscoverして起動します。TopcoatはexperimentalなのでCargo.lockとCLI版を固定します。

03 router・page・route・layout

pageはGETのHTMLを返し、routeはmethodを明示したAPIや処理を返し、layoutはpath prefix配下のpageを包みます。discoverは注釈付きitemをリンク時に収集します。

04 path_param・query_params・Cx

Topcoat 0.5.0では#[path_param]をtuple structへ付け、path_param::<T>(cx)で読みます。queryは#[query_params]付きstructをquery_params::<T>(cx)で遅延parse・memoizeします。

05 view!・component・HTML escaping

view!はHTMLに近い構文へRustの値、if、for、componentを埋めます。文字列値はescapeされるため利用者入力をHTMLとして解釈しません。属性とタグの意味、アクセシビリティも守ります。

06 body extractor・form・JSON・PRG

リクエストbodyは一度だけ読めます。formやJson extractorで型へ変換し、検証後に更新します。HTMLフォーム成功後は303 See OtherでGETへredirectするPRGにより再送信を防ぎます。

07 app_context・request helper・memoize

DB、設定、HTTP clientのように全requestで共有する値はrouterへapp_contextで登録し、Cxから型で借ります。request内のcurrent_userなど繰り返す処理はmemoizeします。

08 assets・CSS・bundle

asset!で参照したファイルはCLIがcompile済みbinaryを走査してcontent hash付きでbundleします。0.5.0の既定出力はtarget/assetsで、routerにAssetBundleをloadします。

09 runtime signal・procedure・shard

signalと\$(...)は初期HTMLをserverで評価し、対応JSをbrowserで再実行します。procedureとshardはserver endpointなので引数を偽装可能と考え、認証・検証・認可を必ず繰り返します。

10 Topcoatエラー・cookie・session・CSRF

router error helperで400、401、403、404、redirectを表し、sessionは生tokenをcookie、hashと期限をDBに持ちます。sessionsはorigin検証も登録します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

methodの意味

STEP 2

status分類

STEP 3

headers

STEP 4

bodyとContent-Type

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

HTTP要求・応答・状態コード

UNIT 01・CONCEPT

HTTP要求・応答・状態コード

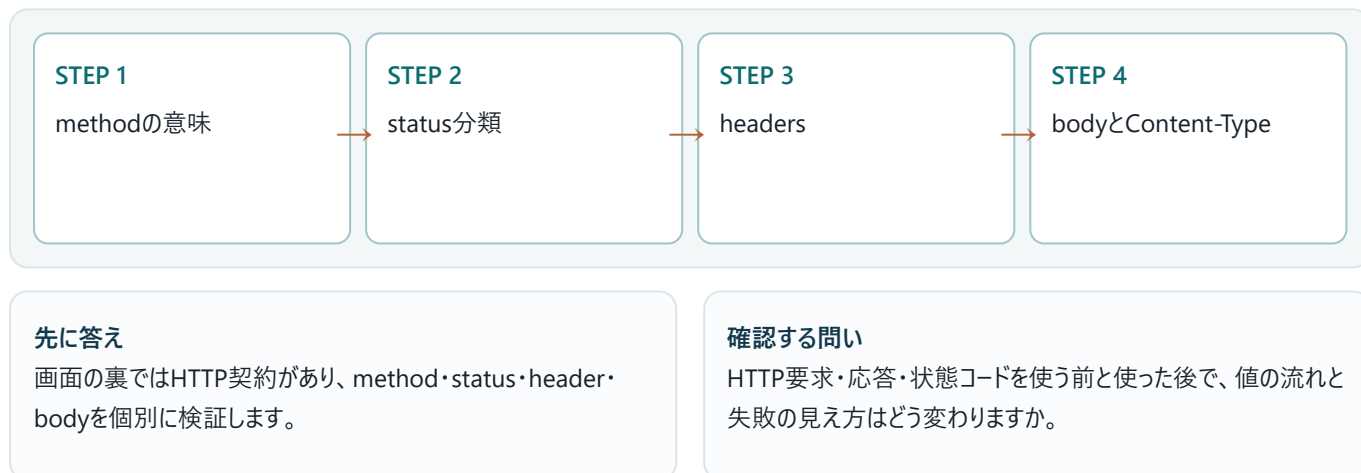
HTTPはmethod、path、headers、bodyの要求と、status、headers、bodyの応答です。GETは安全、PUT/DELETEは冪等など、意味を守るとcache、retry、proxyが正しく働きます。

この単元の到達点

- methodの意味
- status分類
- headers
- bodyとContent-Type

なぜバックエンドで必要か

TopcoatのpageはHTML、routeはAPI値を返します。ハンドラより前後のHTTP契約を理解して404、400、409、500を使い分けます。



UNIT 01・MENTAL MODEL

HTTP要求・応答・状態コードを図でつかむ



HTTPはmethod、path、headers、bodyの要求と、status、headers、bodyの応答です。GETは安全、PUT/DELETEは冪等など、意味を守るとcache、retry、proxyが正しく働きます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	methodの意味	methodの意味 — 入力と前提を確認し、境界を曖昧にしない。
2	status分類	status分類 — 値がどの順で変換されるかを追跡する。
3	headers	headers — 失敗を再現し、型またはログから原因を特定する。
4	bodyとContent-Type	bodyとContent-Type — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

HTTP要求・応答・状態コードとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

HTTP要求・応答・状態コードだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
GET /notes?page=1 HTTP/1.1
Host: localhost:3000
Accept: text/html

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

●●● 実行コマンド

```
curl -i http://127.0.0.1:3000/health
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
GET /notes?page=1 HTTP/1.1	失敗時は早期returnし、呼び出し元へエラーを伝播します。
Host: localhost:3000	この行が受け取る値、作る値、副作用の有無を確認します。
Accept: text/html	この行が受け取る値、作る値、副作用の有無を確認します。
HTTP/1.1 200 OK	この行が受け取る値、作る値、副作用の有無を確認します。
Content-Type: text/html; charset=utf-8	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がmethodの意味の条件を満たすか確認する。
- ✓ status分類を実行し、途中の型と状態を追う。
- ✓ headersで失敗を成功経路から分離する。
- ✓ bodyとContent-Typeにより、出力と副作用を検証する。

型を声に出す HTTP要求・応答・状態コードに関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

HTTP要求・応答・状態コード

DevToolsのNetworkでRequest Method、Status Code、Content-Type、応答HTMLを確認します。

観察ポイント

- methodの意味
- status分類
- headers

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn status(kind:&str)->u16{match kind{"created"=>201,"invalid"=>400,"missing"=>404,"conflict"=>409,"_"=>500}}
fn main(){assert_eq!(status("created"),201);}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

TopcoatのpageはHTML、routeはAPI値を返します。ハンドラより前後のHTTP契約を理解して404、400、409、500を使い分けます。



リクエスト側

- methodの意味を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- status分類を小さな責務へ分割
- headersをResultとログで表現
- bodyとContent-Typeをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	methodの意味に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	status分類の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	headersまたはbodyとContent-Typeの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

HTTP要求・応答・状態コードを再現して切り分ける

HTTP要求・応答・状態コードの問題を、環境・入力・型・副作用の順に切り分けます。

1. STEP 1 — エラー全文と再現コマンドを保存する。対象: methodの意味
2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: status分類
3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: headers
4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: bodyとContent-Type

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	methodの意味	methodの意味 — 入力と前提を確認し、境界を曖昧にしない。
2	status分類	status分類 — 値がどの順で変換されるかを追跡する。
3	headers	headers — 失敗を再現し、型またはログから原因を特定する。
4	bodyとContent-Type	bodyとContent-Type — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 画面の裏ではHTTP契約があり、method・status・header・bodyを個別に検証します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 ・ PRACTICE

手を動かす演習

create、update、delete、validation error、not foundに適切なmethodとstatusを割り当ててください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn status(kind:&str)->u16{match kind{"created"=>201,"invalid"=>400,"missing"=>404,"conflict"=>409,"_"=>500}}
fn main(){assert_eq!(status("created"),201);}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 画面の裏ではHTTP契約があり、method・status・header・bodyを個別に検証します。
- ✓ HTTP要求・応答・状態コードとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

Topcoat 0.5.0最小アプリ

UNIT 02 · CONCEPT

Topcoat 0.5.0最小アプリ

Cargo binaryへtopcoat 0.5.0とTokioを追加し、pageとcomponentをdiscoverして起動します。TopcoatはexperimentalなのでCargo.lockとCLI版を固定します。

この単元の到達点

- Cargo.toml固定
- #[page]
- view!
- Router discover

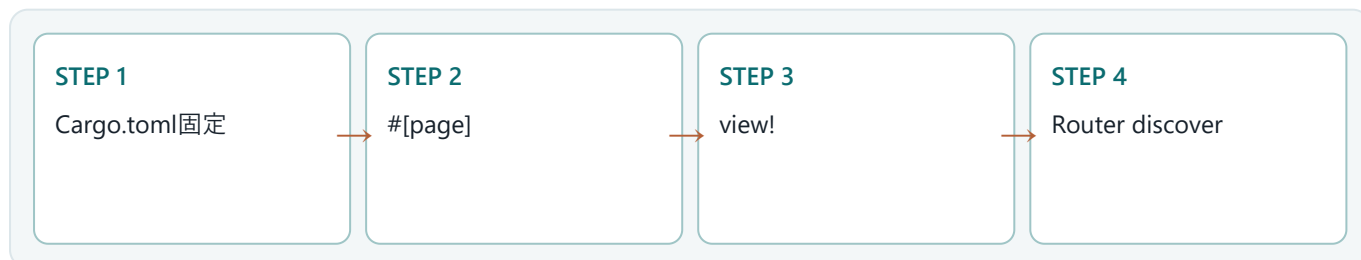
なぜバックエンドで必要か

開発はtopcoat devで再build・asset bundle・restartをまとめます。cargo runだけではassetを使うアプリがbundle未生成で失敗できます。



UNIT 02 · MENTAL MODEL

Topcoat 0.5.0最小アプリを図でつかむ



Cargo binaryへtopcoat 0.5.0とTokioを追加し、pageとcomponentをdiscoverして起動します。TopcoatはexperimentalなのでCargo.lockとCLI版を固定します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Cargo.toml固定	Cargo.toml固定 — 入力と前提を確認し、境界を曖昧にしない。
2	#[page]	#[page] — 値がどの順で変換されるかを追跡する。
3	view!	view! — 失敗を再現し、型またはログから原因を特定する。
4	Router discover	Router discover — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Topcoat 0.5.0最小アプリとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE まず動く最小コード

Topcoat 0.5.0最小アプリだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use topcoat::{Result,router::{Router,RouterBuilderDiscoverExt,page},view::{component,view}};
#[page("/")]  
async fn home()->Result{view!{<!DOCTYPE html><html><body>hello(name:"World")</body></html>}}  
#[component]  
async fn hello(name:&str)->Result{view!{<h1>"Hello, "(name)!"</h1>}}  
#[tokio::main]  
async fn main(){topcoat::start(Router::builder().discover().build()).await.unwrap();}
```

● ● ● 実行コマンド

```
cargo install topcoat-cli --version 0.5.0 --locked  
topcoat dev
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use topcoat::{Result,router:: {Router,RouterBuilderDiscoverExt,page},view::</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[page("/")]async fn home()->Result{view! {<!DOCTYPE html><html><body>hello</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[component]async fn hello(name:&str)- >Result{view!{<h1>"Hello, "(name)"!"</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[tokio::main]async fn main() {topcoat::start(Router::builder().discover().</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力Cargo.toml固定の条件を満たすか確認する。
- ✓ `#[page]`を実行し、途中の型と状態を追う。
- ✓ `view!`で失敗を成功経路から分離する。
- ✓ Router discoverにより、出力と副作用を検証する。

型を声に出す Topcoat 0.5.0最小アプリに関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

Topcoat 0.5.0最小アプリ

RustNoteの見出しとWelcomeカードがサーバー描画されます。

観察ポイント

- Cargo.toml固定
- #[page]
- view!

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[component] async fn card(title:&str, body:&str) -> Result{view!{<article><h2>(title)</h2><p>(body)</p></article>}}
```

```
#[page("/")] async fn home() -> Result{view!{<main><h1>"RustNote"</h1>card(title:"Welcome", body:"Topcoat 0.5.0")</main>}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

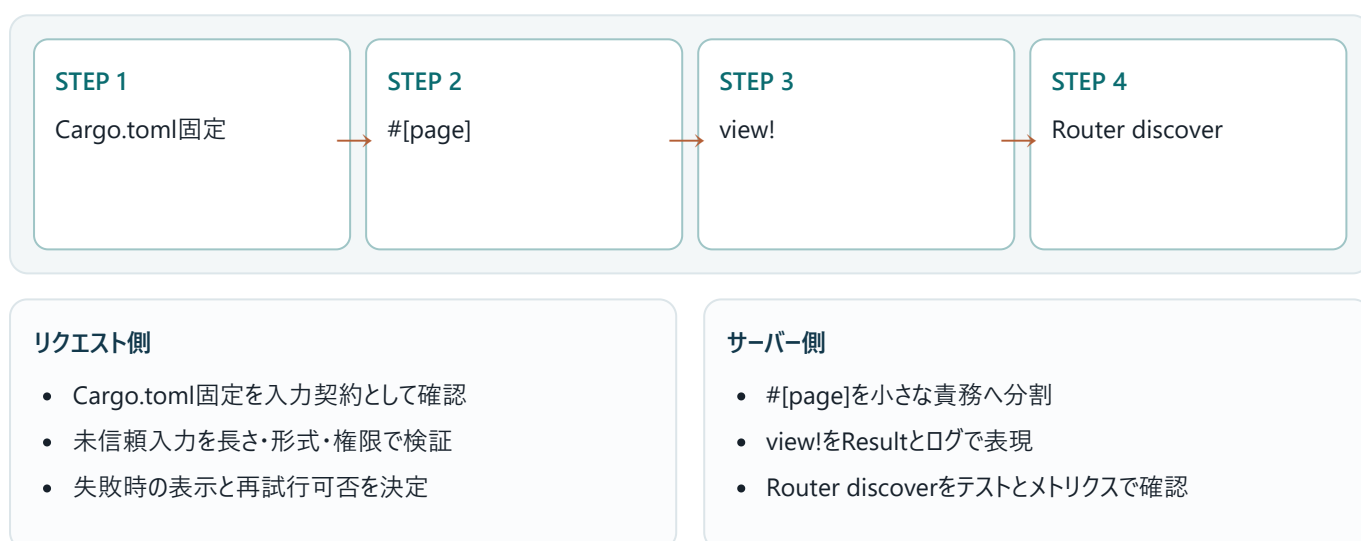
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

開発はtopcoat devで再build・asset bundle・restartをまとめます。cargo runだけではassetを使うアプリがbundle未生成で失敗できます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Cargo.toml固定に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	#[page]の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	view!またはRouter discoverの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

Topcoat 0.5.0最小アプリを再現して切り分ける

Topcoat 0.5.0最小アプリの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Cargo.toml固定
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: #[page]
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: view!
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Router discover

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Cargo.toml固定	Cargo.toml固定 — 入力と前提を確認し、境界を曖昧にしない。
2	#[page]	#[page] — 値がどの順で変換されるかを追跡する。
3	view!	view! — 失敗を再現し、型またはログから原因を特定する。
4	Router discover	Router discover — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 0.5.0のcrate、CLI、API、Cargo.lockを一組で固定します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

見出し、説明、現在時刻を表示するページと一つのcomponentを追加してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[component]async fn card(title:&str,body:&str)->Result{view!{<article><h2>(title)</h2><p>(body)</p></article>}}  
#[page("/")]async fn home()->Result{view!{<main><h1>"RustNote"</h1>card(title:"Welcome",body:"Topcoat 0.5.0")</main>}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 0.5.0のcrate、CLI、API、Cargo.lockを一組で固定します。
- ✓ Topcoat 0.5.0最小アプリとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

router・page・route・layout

UNIT 03 ・ CONCEPT

router・page・route・layout

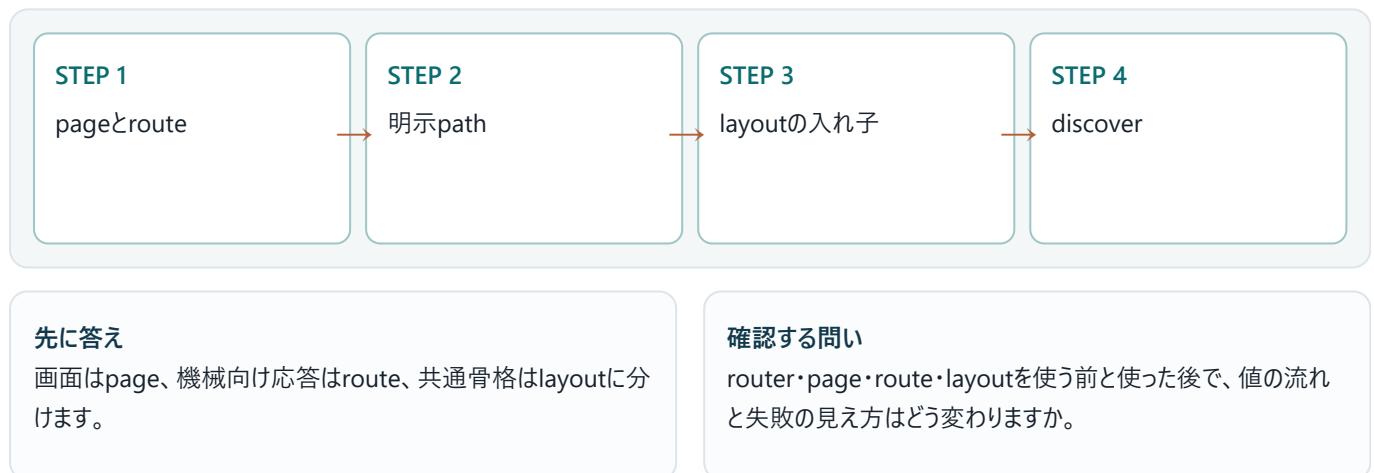
pageはGETのHTMLを返し、routeはmethodを明示したAPIや処理を返し、layoutはpath prefix配下のpageを包みます。discoverは注釈付きitemをリンク時に収集します。

この単元の到達点

- pageとroute
- 明示path
- layoutの入れ子
- discover

なぜバックエンドで必要か

URL構造をmoduleへ対応させ、共有HTMLはlayout、機械向け応答はroute、利用者向け画面はpageへ分けます。



UNIT 03 ・ MENTAL MODEL

router・page・route・layoutを図でつかむ



pageはGETのHTMLを返し、routeはmethodを明示したAPIや処理を返し、layoutはpath prefix配下のpageを包みます。discoverは注釈付きitemをリンク時に収集します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	pageとroute	pageとroute — 入力と前提を確認し、境界を曖昧にしない。
2	明示path	明示path — 値がどの順で変換されるかを追跡する。
3	layoutの入れ子	layoutの入れ子 — 失敗を再現し、型またはログから原因を特定する。
4	discover	discover — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

router・page・route・layoutとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03・MINIMUM CODE まず動く最小コード

router・page・route・layoutだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[layout("/")]async fn root(slot:Result)->Result{view!{<!DOCTYPE html><html><body><nav><a href="/">"Home"
</a></nav><(slot?)</body></html>}}
#[page("/")]async fn home()->Result{view!{<h1>"Home"</h1>}}
#[route(GET "/api/health")]async fn health()->Result<&'static str>{Ok("ok")}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[layout("/")async fn root(slot:Result)->Result{view!{<!DOCTYPE html><htm</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[page("/")async fn home()->Result{view!{<h1>"Home"</h1>}}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[route(GET "/api/health")]async fn health()->Result<&'static str>{Ok("ok"</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力pageとrouteの条件を満たすか確認する。
- ✓ 明示pathを実行し、途中の型と状態を追う。
- ✓ layoutの入れ子で失敗を成功経路から分離する。
- ✓ discoverにより、出力と副作用を検証する。

型を声に出す router・page・route・layoutに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

router・page・route・layout

共通ナビゲーションの下にページ固有の見出しが表示され、healthは本文okだけを返します。

観察ポイント

- pageとroute
- 明示path
- layoutの入れ子

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[page("/notes")]async fn notes()->Result{view!{{<h1>"Notes"</h1>}}}
#[route(GET "/api/health")]async fn health()->Result<&'static str>{Ok("ok")}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

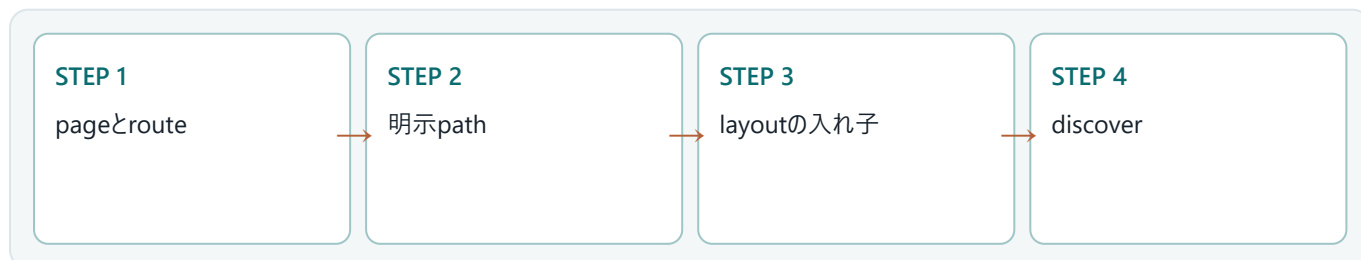
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 ・ BACKEND APPLICATION

バックエンドのどこで使うか

URL構造をmoduleへ対応させ、共有HTMLはlayout、機械向け応答はroute、利用者向け画面はpageへ分けます。



リクエスト側

- pageとrouteを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 明示pathを小さな責務へ分割
- layoutの入れ子をResultとログで表現
- discoverをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	pageとrouteに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	明示pathの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	layoutの入れ子またはdiscoverの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

router・page・route・layoutを再現して切り分ける

router・page・route・layoutの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: pageとroute
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 明示path
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: layoutの入れ子
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: discover

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	pageとroute	pageとroute — 入力と前提を確認し、境界を曖昧にしない。
2	明示path	明示path — 値がどの順で変換されるかを追跡する。
3	layoutの入れ子	layoutの入れ子 — 失敗を再現し、型またはログから原因を特定する。
4	discover	discover — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 画面はpage、機械向け応答はroute、共通骨格はlayoutに分けます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

/notesと/notes/{note_id}のpage、/api/healthのrouteを登録してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[page("/notes")]async fn notes()->Result{view!{<h1>"Notes"</h1>}}
#[route(GET "/api/health")]async fn health()->Result<&'static str>{Ok("ok")}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 画面はpage、機械向け応答はroute、共通骨格はlayoutに分けます。
- ✓ router・page・route・layoutとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

path_param・query_params・Cx

UNIT 04・CONCEPT

path_param・query_params・Cx

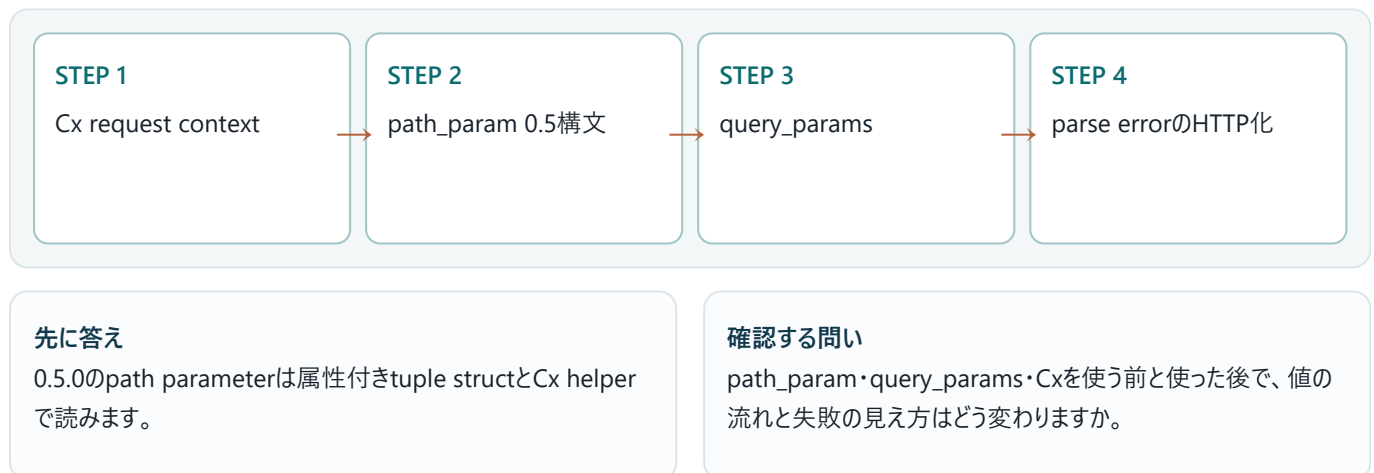
Topcoat 0.5.0では#[path_param]をtuple structへ付け、path_param::<T>(cx)で読みます。queryは#[query_params]付きstructをquery_params::<T>(cx)で遅延parse・memoizeします。

この単元の到達点

- Cx request context
- path_param 0.5構文
- query_params
- parse errorのHTTP化

なぜバックエンドで必要か

IDと検索条件は未信頼入力です。型parse後も正数、上限、文字数、認可を検証し、SQLへparameter bindします。



UNIT 04・MENTAL MODEL

path_param・query_params・Cxを図でつかむ



Topcoat 0.5.0では#[path_param]をtuple structへ付け、path_param::<T>(cx)で読みます。queryは#[query_params]付きstructをquery_params::<T>(cx)で遅延parse・memoizeします。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Cx request context	Cx request context — 入力と前提を確認し、境界を曖昧にしない。
2	path_param 0.5構文	path_param 0.5構文 — 値がどの順で変換されるかを追跡する。
3	query_params	query_params — 失敗を再現し、型またはログから原因を特定する。
4	parse errorのHTTP化	parse errorのHTTP化 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

path_param・query_params・Cxとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE まず動く最小コード

path_param・query_params・Cxだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[path_param(error = bad_request)]struct NoteId(i64);
#[query_params(error = bad_request)]struct NotesQuery{page:Option<u32>,q:Option<String>}
#[page("/notes/{note_id}")]async fn show(cx:&Cx)->Result{let id=path_param::<NoteId>(cx)?;view!{<h1>"Note "
(id.0)</h1>}}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[path_param(error = bad_request)]struct NoteId(i64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[query_params(error = bad_request)]struct NotesQuery{page:Option<u32>,q:O</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[page("/{notes}/{note_id}")]async fn show(cx:&Cx)->Result{let id=path_param</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力が入力 context の条件を満たすか確認する。
- ✓ `path_param` 0.5 構文を実行し、途中の型と状態を追う。
- ✓ `query_params` で失敗を成功経路から分離する。
- ✓ `parse error` の HTTP 化により、出力と副作用を検証する。

型を声に出す `path_param・query_params・Cx` に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
GET /notes/7 → Note 7
GET /notes/not-number → 400 Bad Request
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn normalize(page:Option<u32>,q:Option<String>)->(u32,Option<String>){let
page=page.unwrap_or(1).clamp(1,1000);let q=q.map(|s|s.trim().to_owned()).filter(|s|!s.is_empty());(page,q)}
fn main(){assert_eq!(normalize(Some(0),Some(" ".into())),(1,None));}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

IDと検索条件は未信頼入力です。型parse後も正数、上限、文字数、認可を検証し、SQLへparameter bindします。



リクエスト側

- Cx request contextを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- path_param 0.5構文を小さな責務へ分割
- query_paramsをResultとログで表現
- parse errorのHTTP化をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Cx request contextに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	path_param 0.5構文の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	query_paramsまたはparse errorのHTTP化の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
GET /notes/abc
HTTP/1.1 400 Bad Request
path parameter `note_id` could not be parsed
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

path_param・query_params・Cxを再現して切り分ける

path_param・query_params・Cxの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Cx request context
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: path_param 0.5構文
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: query_params
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: parse errorのHTTP化

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Cx request context	Cx request context — 入力と前提を確認し、境界を曖昧にしない。
2	path_param 0.5構文	path_param 0.5構文 — 値がどの順で変換されるかを追跡する。
3	query_params	query_params — 失敗を再現し、型またはログから原因を特定する。
4	parse errorのHTTP化	parse errorのHTTP化 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 0.5.0のpath parameterは属性付きtuple structとCx helperで読みます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

pageが0なら1、1000超なら1000へ制限し、qはtrimして空ならNoneにしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn normalize(page:Option<u32>,q:Option<String>)->(u32,Option<String>){let
page=page.unwrap_or(1).clamp(1,1000);let q=q.map(|s|s.trim().to_owned()).filter(|s|!s.is_empty());(page,q)}
fn main(){assert_eq!(normalize(Some(0),Some(" ".into())),(1,None));}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 0.5.0のpath parameterは属性付きtuple structとCx helperで読みます。
- ✓ path_param・query_params・Cxとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

view!・component・HTML escaping

UNIT 05・CONCEPT

view!・component・HTML escaping

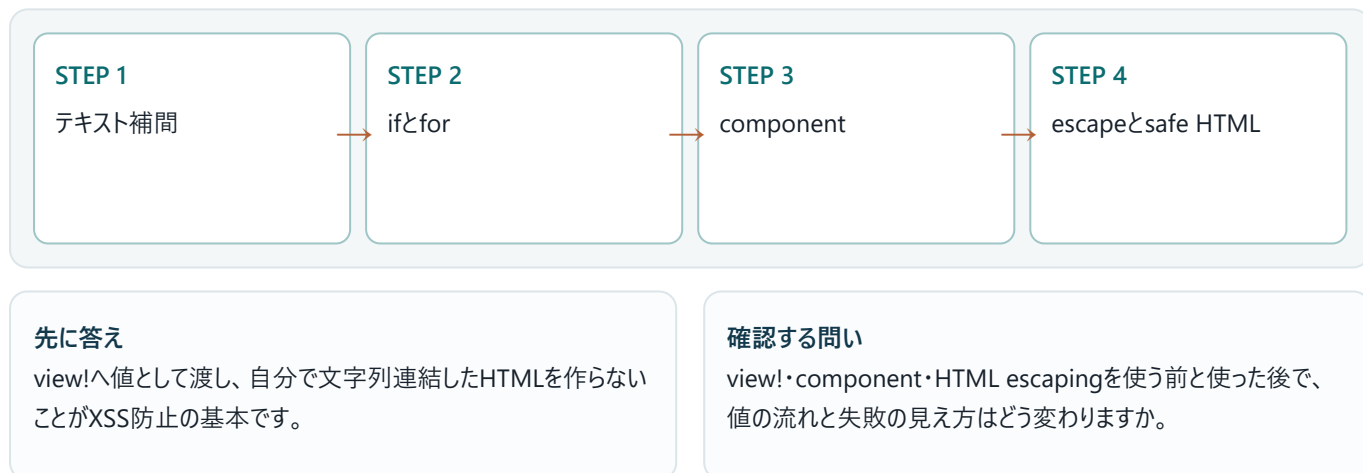
view!はHTMLに近い構文へRustの値、if、for、componentを埋めます。文字列値はescapeされるため利用者入力をHTMLとして解釈しません。属性とタグの意味、アクセシビリティも守ります。

この単元の到達点

- テキスト補間
- ifとfor
- component
- escapeとsafe HTML

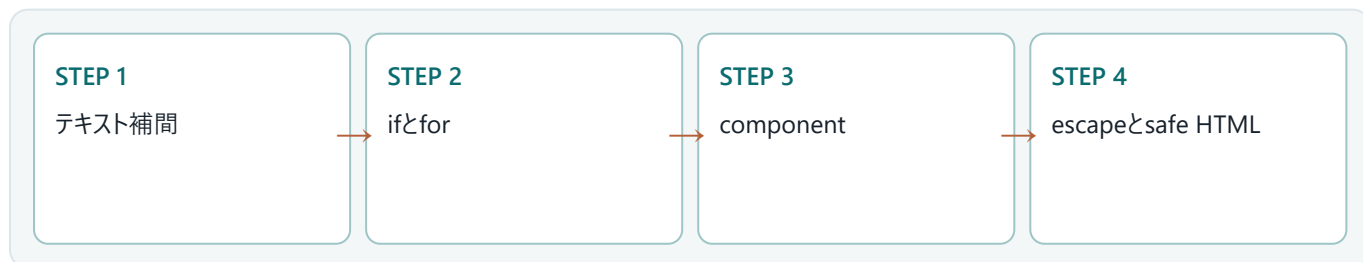
なぜバックエンドで必要か

SSR出力は最初からHTMLとして届きます。利用者入力をraw HTMLへ変換せず、URL、属性、CSS classも文脈に応じて制約します。



UNIT 05・MENTAL MODEL

view!・component・HTML escapingを図でつかむ



view!はHTMLに近い構文へRustの値、if、for、componentを埋めます。文字列値はescapeされるため利用者入力をHTMLとして解釈しません。属性とタグの意味、アクセシビリティも守ります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	テキスト補間	テキスト補間 — 入力と前提を確認し、境界を曖昧にしない。
2	ifとfor	ifとfor — 値がどの順で変換されるかを追跡する。
3	component	component — 失敗を再現し、型またはログから原因を特定する。
4	escapeとsafe HTML	escapeとsafe HTML — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

view!・component・HTML escapingとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE まず動く最小コード

view!・component・HTML escapingだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[component]async fn note_item(title:&str,done:bool)->Result{view!{<li class=if done{"done"}else{"pending"}>
<span>(title)</span></li>}}
#[page("/notes")]async fn notes()->Result{let titles=["<script>alert(1)</script>", "Rust"];view!{<ul>for
title in titles{note_item(title,done:false)}</ul>}}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[component]async fn note_item(title:&str,done:bool)->Result{view! {<li cla</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[page("/notes")]async fn notes()->Result{let titles=["<script>alert(1)</s</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>view!・component・HTML escaping</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力テキスト補間の条件を満たすか確認する。
- ✓ ifとforを実行し、途中の型と状態を追う。
- ✓ componentで失敗を成功経路から分離する。
- ✓ escapeとsafe HTMLにより、出力と副作用を検証する。

型を声に出す view!・component・HTML escapingに関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

view!・component・HTML escaping

入力でscriptタグ文字列が含まれていてもテキストとして表示され、Noteカードと空状態が読みやすく描画されます。

観察ポイント

- テキスト補間
- ifとfor
- component

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[component] async fn note_card(title:&str,url:&str)->Result{view!{<article><h2><a href=(url)>(title)</a></h2></article>}}
#[component] async fn list(items:&[(i64,String)])->Result{view!{if items.is_empty(){<p>"まだノートがありません"</p>}else{for(id,title)in items{note_card(title,url:&format!("{}",id))}}}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

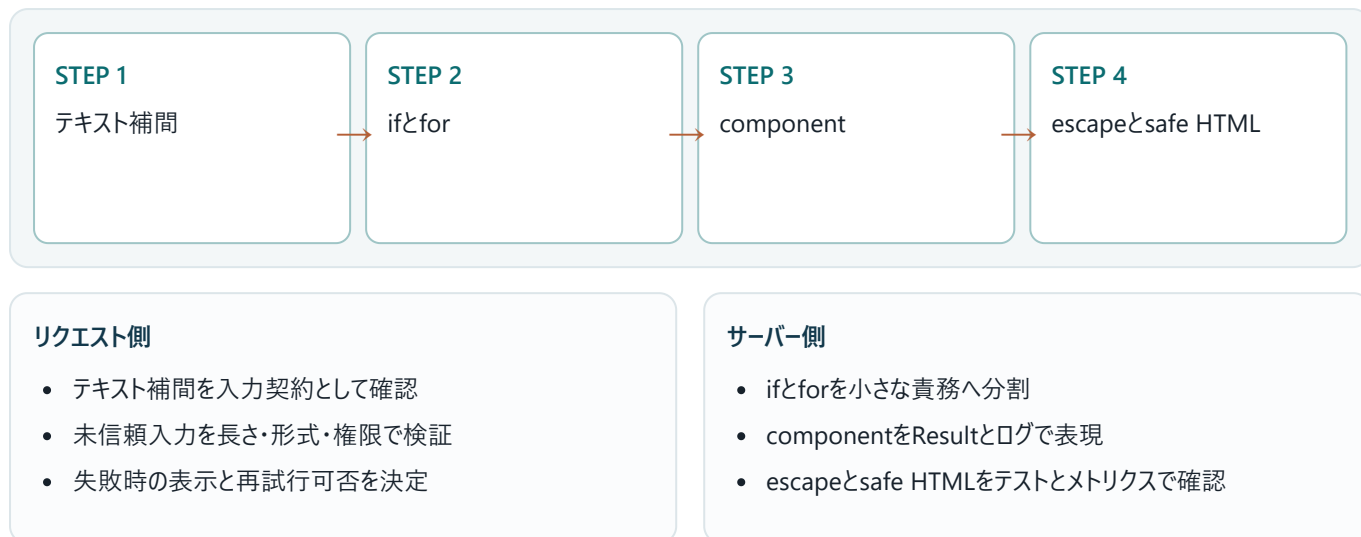
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05・BACKEND APPLICATION

バックエンドのどこで使うか

SSR出力は最初からHTMLとして届きます。利用者入力をraw HTMLへ変換せず、URL、属性、CSS classも文脈に応じて制約します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	テキスト補間に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	ifとforの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	componentまたはescapeとsafe HTMLの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

view!・component・HTML escapingを再現して切り分ける

view!・component・HTML escapingの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: テキスト補間
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: ifとfor
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: component
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: escapeとsafe HTML

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	テキスト補間	テキスト補間 — 入力と前提を確認し、境界を曖昧にしない。
2	ifとfor	ifとfor — 値がどの順で変換されるかを追跡する。
3	component	component — 失敗を再現し、型またはログから原因を特定する。
4	escapeとsafe HTML	escapeとsafe HTML — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 view!へ値として渡し、自分で文字列連結したHTMLを作らないことがXSS防止の基本です。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

NoteCard componentへtitle、done、detail URLを渡し、空リスト時のメッセージも表示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[component]async fn note_card(title:&str,url:&str)->Result{view!{<article><h2><a href={url}>{title}</a></h2></article>}}
#[component]async fn list(items:&[(i64,String)])->Result{view!{if items.is_empty(){<p>"まだノートがありません"</p>}else{for(id,title)in items{note_card(title,url:&format!("{}",notes/{id}))}}}}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ view!へ値として渡し、自分で文字列連結したHTMLを作らないことがXSS防止の基本です。
- ✓ view!・component・HTML escapingとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

body extractor・form・JSON・PRG

UNIT 06 ・ CONCEPT

body extractor・form・JSON・PRG

リクエストbodyは一度だけ読めます。formやJson extractorで型へ変換し、検証後に更新します。HTMLフォーム成功後は303 See OtherでGETへredirectするPRGにより再送信を防ぎます。

この単元の到達点

- bodyは一度
- typed extractor
- 入力検証
- POST-Redirect-GET

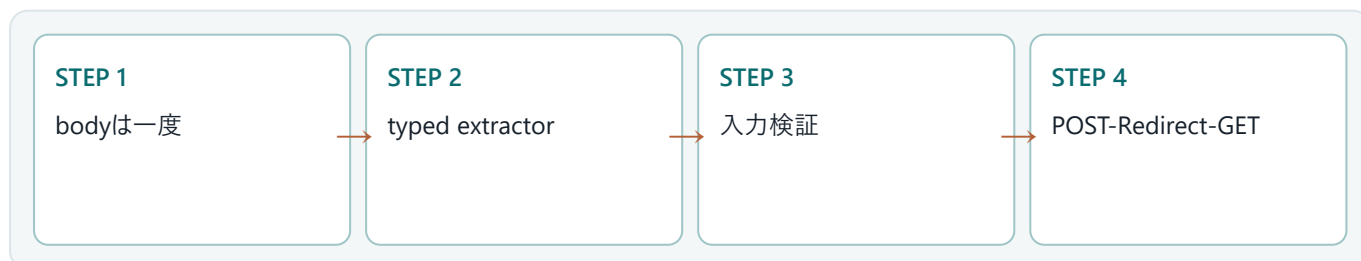
なぜバックエンドで必要か

検証エラーでは入力値を保持したフォームを400で再表示し、成功時だけDBを更新してredirectします。二重送信には一意制約やidempotencyも使います。



UNIT 06 ・ MENTAL MODEL

body extractor・form・JSON・PRGを図でつかむ



リクエストbodyは一度だけ読めます。formやJson extractorで型へ変換し、検証後に更新します。HTMLフォーム成功後は303 See OtherでGETへredirectするPRGにより再送信を防ぎます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の

型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	bodyは一度	bodyは一度 — 入力と前提を確認し、境界を曖昧にしない。
2	typed extractor	typed extractor — 値がどの順で変換されるかを追跡する。
3	入力検証	入力検証 — 失敗を再現し、型またはログから原因を特定する。
4	POST-Redirect-GET	POST-Redirect-GET — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

body extractor・form・JSON・PRGとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE

まず動く最小コード

body extractor・form・JSON・PRGだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(serde::Deserialize)]struct CreateNote{title:String}
#[route(POST "/notes")]async fn create(Form(input):Form<CreateNote>)->Result<SeeOther>{let
title=input.title.trim();if title.is_empty(){return
Err(bad_request().into());}save(title).await?;Ok(see_other("/notes"))}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(serde::Deserialize)]struct CreateNote{title:String}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[route(POST "/notes")]async fn create(Form(input):Form<CreateNote>)->Resu</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>body extractor・form・JSON・PRG</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力bodyは一度の条件を満たすか確認する。
- ✓ typed extractorを実行し、途中の型と状態を追う。
- ✓ 入力検証で失敗を成功経路から分離する。
- ✓ POST-Redirect-GETにより、出力と副作用を検証する。

型を声に出す body extractor・form・JSON・PRGに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

body extractor・form・JSON・PRG

送信後URLは/notesへ戻り、新しいカードが一度だけ表示されます。エラー時は入力値と項目別メッセージが残ります。

観察ポイント

- bodyは一度
- typed extractor
- 入力検証

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn validate(title:&str,body:&str)->Result<(String,String),&'static str>{let t=title.trim();if t.is_empty()||t.chars().count()>80{return Err("invalid title");}if body.chars().count()>5000{return Err("body too long");}Ok((t.into(),body.into()))}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

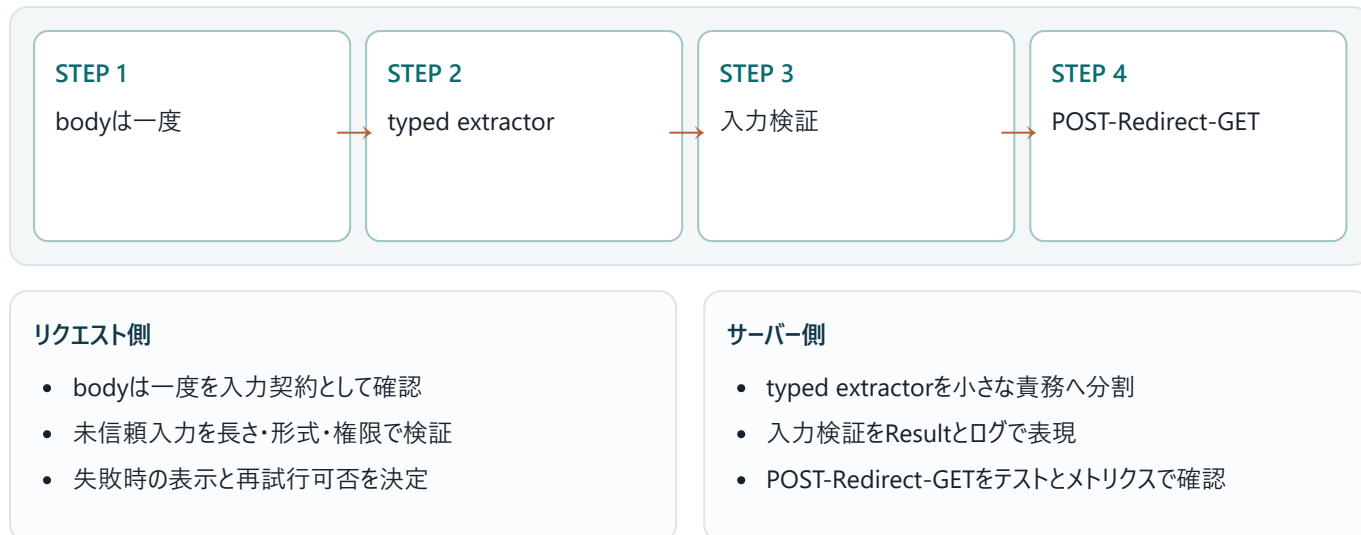
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06・BACKEND APPLICATION

バックエンドのどこで使うか

検証エラーでは入力値を保持したフォームを400で再表示し、成功時だけDBを更新してredirectします。二重送信には一意制約やidempotencyも使います。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	bodyは一度に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	typed extractorの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	入力検証またはPOST-Redirect-GETの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

body extractor・form・JSON・PRGを再現して切り分ける

body extractor・form・JSON・PRGの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: bodyは一度
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: typed extractor
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 入力検証
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: POST-Redirect-GET

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	bodyは一度	bodyは一度 — 入力と前提を確認し、境界を曖昧にしない。
2	typed extractor	typed extractor — 値がどの順で変換されるかを追跡する。
3	入力検証	入力検証 — 失敗を再現し、型またはログから原因を特定する。
4	POST-Redirect-GET	POST-Redirect-GET — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 bodyを型へparseし、副作用前に検証し、成功後は303でGETへ戻します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

titleを1～80文字、bodyを5000文字以下で検証し、成功時303、失敗時400にしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn validate(title:&str,body:&str)->Result<(String,String),&'static str>{let t=title.trim();if t.is_empty()||t.chars().count(>80){return Err("invalid title")}if body.chars().count(>5000){return Err("body too long")}Ok((t.into(),body.into()))}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ bodyを型へparseし、副作用前に検証し、成功後は303でGETへ戻します。
- ✓ body extractor・form・JSON・PRGとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

app_context・request helper・memoize

UNIT 07・CONCEPT

app_context・request helper・memoize

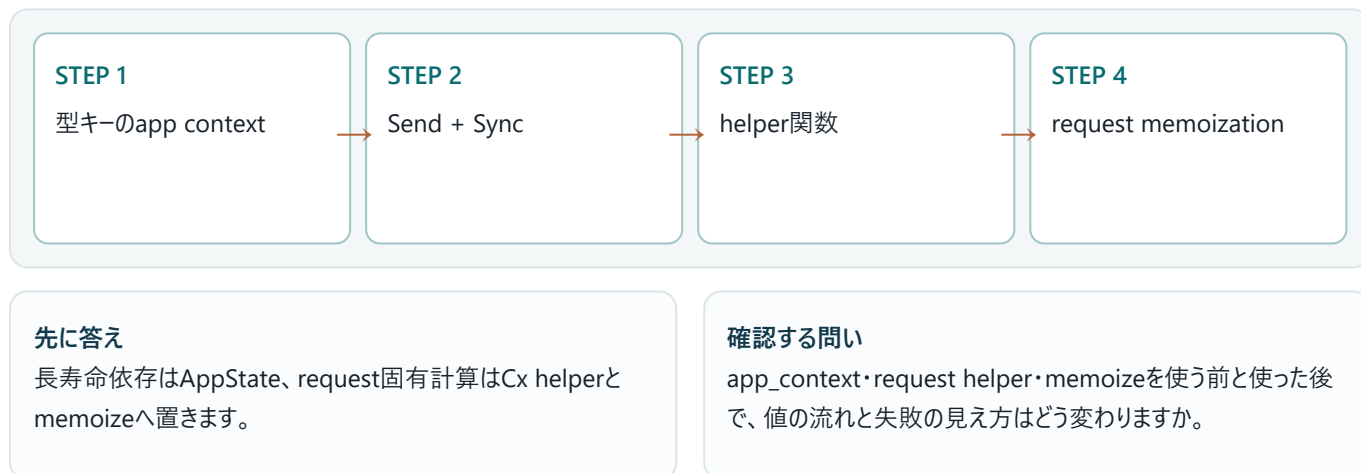
DB、設定、HTTP clientのように全requestで共有する値はrouterへapp_contextで登録し、Cxから型で借ります。request内のcurrent_userなど繰り返す処理はmemoizeします。

この単元の到達点

- 型キーのapp context
- Send + Sync
- helper関数
- request memoization

なぜバックエンドで必要か

AppStateは長寿命でSend+Syncなhandleだけを持ち、リクエスト固有の入力やロックguardを保存しません。



UNIT 07・MENTAL MODEL

app_context・request helper・memoizeを図でつかむ



DB、設定、HTTP clientのように全requestで共有する値はrouterへapp_contextで登録し、Cxから型で借ります。request内のcurrent_userなど繰り返す処理はmemoizeします。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	型キーのapp context	型キーのapp context — 入力と前提を確認し、境界を曖昧にしない。
2	Send + Sync	Send + Sync — 値がどの順で変換されるかを追跡する。
3	helper関数	helper関数 — 失敗を再現し、型またはログから原因を特定する。
4	request memoization	request memoization — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

app_context・request helper・memoizeとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07・MINIMUM CODE

まず動く最小コード

app_context・request helper・memoizeだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
struct AppState{db:Database}
fn db(cx:&Cx)->&Database{&app_context::<AppState>(cx).db}
#[memoize]async fn current_user(cx:&Cx)->Result<Option<User>>{let Some(hash)=session::token_hash(cx).await?
else{return Ok(None)};db(cx).find_user_by_session(&hash).await}
let router=Router::builder().discover().app_context(state).build();
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>struct AppState{db:Database}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn db(cx:&Cx)->&Database{&app_context::<AppState>(cx).db}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[memoize]async fn current_user(cx:&Cx)->Result<Option<User>>{let Some(has</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let router=Router::builder().discover().app_context(state).build();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力型が型キーのapp contextの条件を満たすか確認する。
- ✓ Send + Syncを実行し、途中の型と状態を追う。
- ✓ helper関数で失敗を成功経路から分離する。
- ✓ request memoizationにより、出力と副作用を検証する。

型を声に出す app_context・request helper・memoizeに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

同じrequestでlayoutとpageがcurrent_userを呼んでもDB lookupは一回です。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct Config{app_name:String}struct AppState{config:Config}fn config(cx:&Cx)->&Config{&app_context::<AppState>(cx).config}#[page("/about")]async fn about(cx:&Cx)->Result{view!{{h1}>(&config(cx).app_name)</h1>}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

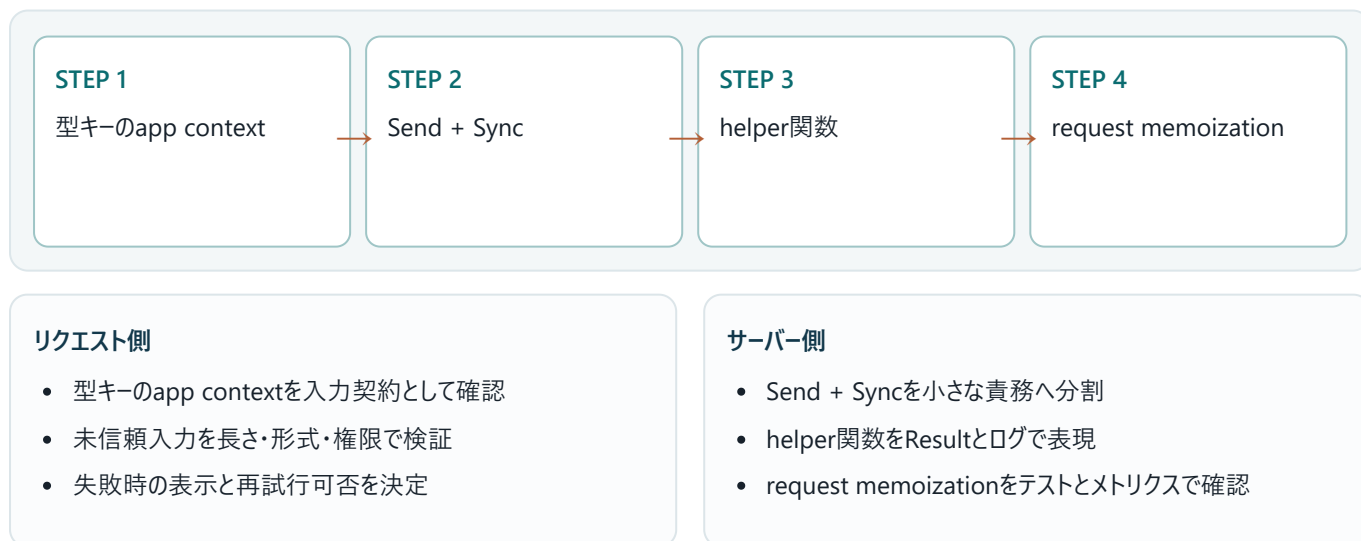
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

AppStateは長寿命でSend+Syncなhandleだけを持ち、リクエスト固有の入力やロックguardを保存しません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	型キーのapp contextに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	Send + Syncの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	helper関数またはrequest memoizationの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

app_context・request helper・memoizeを再現して切り分ける

app_context・request helper・memoizeの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 型キーのapp context
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: Send + Sync
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: helper関数
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: request memoization

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	型キーのapp context	型キーのapp context — 入力と前提を確認し、境界を曖昧にしない。
2	Send + Sync	Send + Sync — 値がどの順で変換されるかを追跡する。
3	helper関数	helper関数 — 失敗を再現し、型またはログから原因を特定する。
4	request memoization	request memoization — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 長寿命依存はAppState、request固有計算はCx helperとmemoizeへ置きます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

ClockとConfigをAppStateへ登録し、handlerからhelper経由で読む形にしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct Config{app_name:String}struct AppState{config:Config}fn config(cx:&Cx)->&Config{&app_context::<AppState>(cx).config}#[page("/about")]async fn about(cx:&Cx)->Result{view!{<h1>(&config(cx).app_name)</h1>}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 長寿命依存はAppState、request固有計算はCx helperとmemoizeへ置きます。
- ✓ app_context・request helper・memoizeとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

assets・CSS・bundle

UNIT 08・CONCEPT

assets・CSS・bundle

asset!で参照したファイルはCLIがcompile済みbinaryを走査してcontent hash付きでbundleします。0.5.0の既定出力はtarget/assetsで、routerにAssetBundleをloadします。

この単元の到達点

- asset!宣言
- topcoat dev
- release build後bundle
- target/assets配布

なぜバックエンドで必要か

binaryと同じsourceからbundleしたtarget/assetsをコンテナの/app/assetsへコピーします。古いbundleとの不一致をCIで防ぎます。



UNIT 08・MENTAL MODEL

assets・CSS・bundleを図でつかむ



asset!で参照したファイルはCLIがcompile済みbinaryを走査してcontent hash付きでbundleします。0.5.0の既定出力はtarget/assetsで、routerにAssetBundleをloadします。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	asset!宣言	asset!宣言 — 入力と前提を確認し、境界を曖昧にしない。
2	topcoat dev	topcoat dev — 値がどの順で変換されるかを追跡する。
3	release build後bundle	release build後bundle — 失敗を再現し、型またはログから原因を特定する。
4	target/assets配布	target/assets配布 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

assets・CSS・bundleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE

まず動く最小コード

assets・CSS・bundleだけに注目の最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
const CSS:Asset=asset!(("./assets/app.css"));
#[layout("/")]  
async fn root(slot:Result)->Result{view!{<html><head><link rel="stylesheet" href=(CSS)></head><body>(slot?)</body></html>}}  
let router=Router::builder().discover().assets(AssetBundle::load().unwrap()).build();
```

● ● ● 実行コマンド

```
cargo build --locked --release  
topcoat asset bundle --release
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>const CSS:Asset=asset!(\"./assets/app.css\");</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[layout(\"/\")]async fn root(slot:Result)->Result{view!{<html> <head> <link r</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let router=Router::builder().discover().assets(AssetBundle::load()).unwrap(</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力が`asset!`宣言の条件を満たすか確認する。
- ✓ `topcoat dev`を実行し、途中の型と状態を追う。
- ✓ `release build`後`bundle`で失敗を成功経路から分離する。
- ✓ `target/assets`配布により、出力と副作用を検証する。

型を声に出す `assets・CSS・bundle`に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

assets・CSS・bundle

整形されたRustNote画面と、Network上のcontent hash付きCSS URLが確認できます。

観察ポイント

- `asset!`宣言
- `topcoat dev`
- `release build`後`bundle`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
cargo build --locked --release
topcoat asset bundle --release
# Docker: COPY --from=build /app/target/assets /app/assets
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

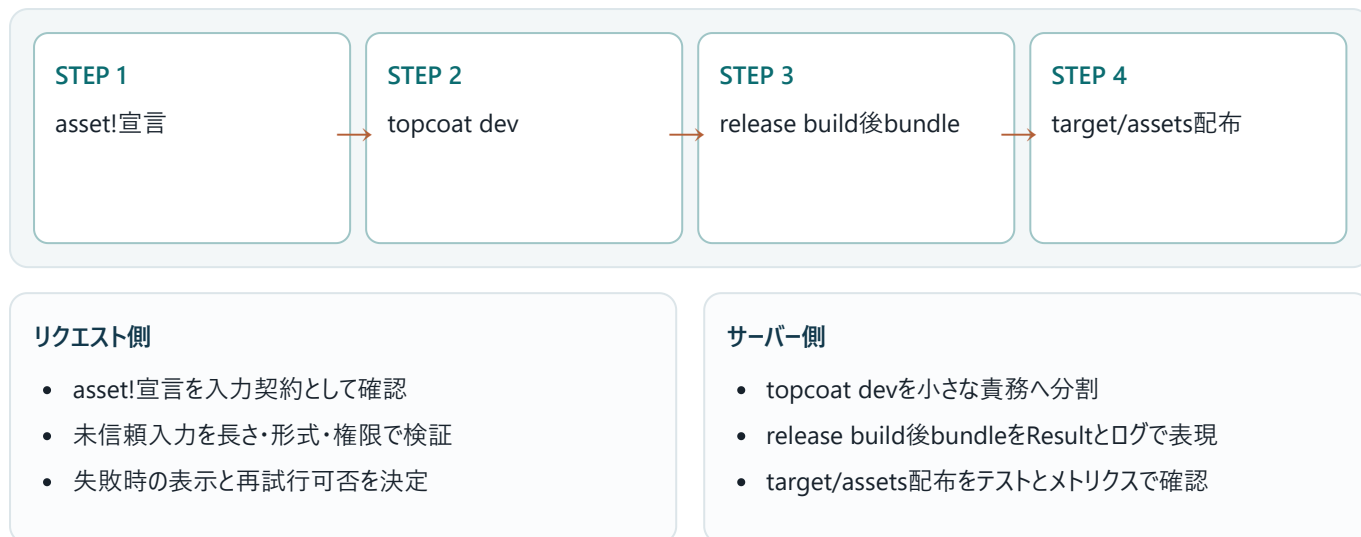
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08・BACKEND APPLICATION

バックエンドのどこで使うか

binaryと同じsourceからbundleしたtarget/assetsをコンテナの/app/assetsへコピーします。古いbundleとの不一致をCIで防ぎます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	asset!宣言に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	topcoat devの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	release build後bundleまたはtarget/assets配布の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08 · DEBUGGING

assets・CSS・bundleを再現して切り分ける

assets・CSS・bundleの問題を、環境・入力・型・副作用の順に切り分けます。

1. STEP 1 — エラー全文と再現コマンドを保存する。対象: asset!宣言
2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: topcoat dev
3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: release build後bundle
4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: target/assets配布

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	asset!宣言	asset!宣言 — 入力と前提を確認し、境界を曖昧にしない。
2	topcoat dev	topcoat dev — 値がどの順で変換されるかを追跡する。
3	release build後bundle	release build後bundle — 失敗を再現し、型またはログから原因を特定する。
4	target/assets配布	target/assets配布 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 0.5.0ではrelease binaryを先にbuildし、target/assetsをbinaryと一緒に配布します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

CSSの背景色を変え、topcoat devでhash URLが変わることをNetwork画面で確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
cargo build --locked --release
topcoat asset bundle --release
# Docker: COPY --from=build /app/target/assets /app/assets
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 0.5.0ではrelease binaryを先にbuildし、target/assetsをbinaryと一緒に配布します。
- ✓ assets・CSS・bundleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

runtime signal・procedure・shard

UNIT 09・CONCEPT

runtime signal・procedure・shard

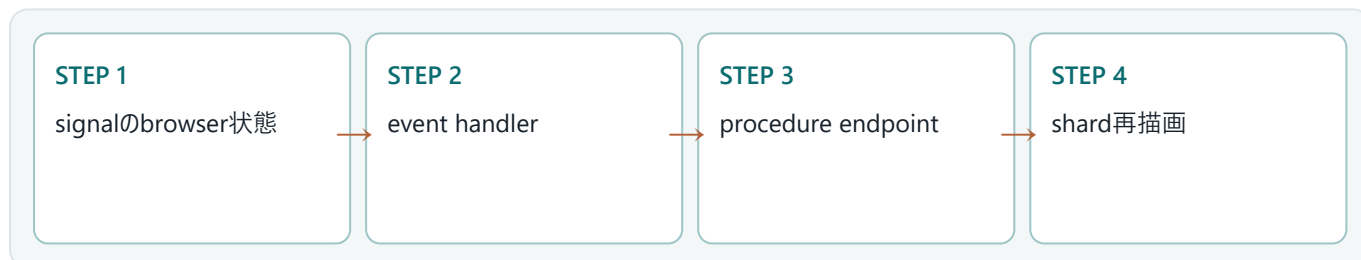
signalと\$(...)は初期HTMLをserverで評価し、対応JSをbrowserで再実行します。procedureとshardはserver endpointなので引数を偽装可能と考え、認証・検証・認可を必ず繰り返します。

この単元の到達点

- signalのbrowser状態
- event handler
- procedure endpoint
- shard再描画

なぜバックエンドで必要か

UI上で隠してもprocedureは直接呼べます。user_idを引数から信用せずsessionから決定し、値の範囲と対象所有者をserverで検証します。



先に答え

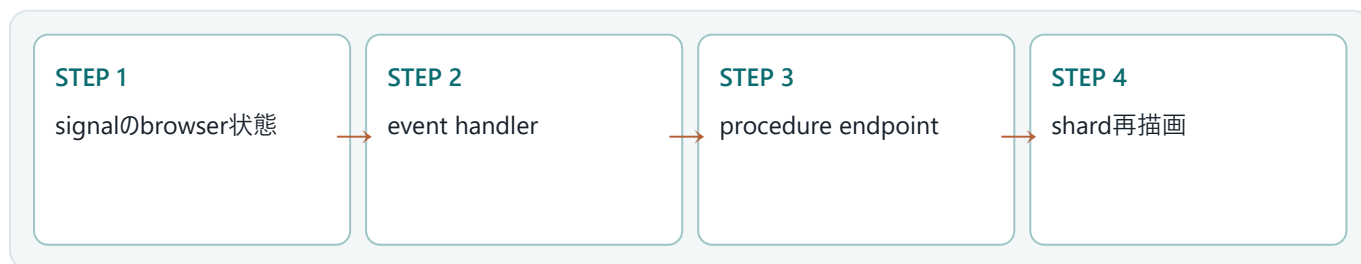
browser反応とserver処理の境界を区別し、procedure/shardを公開APIとして守ります。

確認する問い

runtime signal・procedure・shardを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 09・MENTAL MODEL

runtime signal・procedure・shardを図でつかむ



signalと\$(...)は初期HTMLをserverで評価し、対応JSをbrowserで再実行します。procedureとshardはserver endpointなので引数を偽装可能と考え、認証・検証・認可を必ず繰り返します。図では左から右へ処理を追います。各箱の入口では入力 of 型と信頼度、出口では

結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	signalのbrowser状態	signalのbrowser状態 — 入力と前提を確認し、境界を曖昧にしない。
2	event handler	event handler — 値がどの順で変換されるかを追跡する。
3	procedure endpoint	procedure endpoint — 失敗を再現し、型またはログから原因を特定する。
4	shard再描画	shard再描画 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

runtime signal・procedure・shardとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09 · MINIMUM CODE

まず動く最小コード

runtime signal・procedure・shardだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[component]async fn counter()->Result{view!{signal count=0.0;<button @click=${_e|count.increment()}>"+1"
</button><p>"Count: "${count.get()}</p>}}
#[procedure]async fn double(value:f64)->Result<f64>{Ok(value*2.0)}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[component]async fn counter()->Result{view! {signal count=0.0;<button @cli</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[procedure]async fn double(value:f64)- >Result<f64>{Ok(value*2.0)}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>runtime signal・procedure・shard</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がsignalのbrowser状態の条件を満たすか確認する。
- ✓ event handlerを実行し、途中の型と状態を追う。
- ✓ procedure endpointで失敗を成功経路から分離する。
- ✓ shard再描画により、出力と副作用を検証する。

型を声に出す runtime signal・procedure・shardに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

runtime signal・procedure・shard

入力に応じて結果部分だけが差し替わります。長すぎる検索語はserverが400で拒否します。

観察ポイント

- signalのbrowser状態
- event handler
- procedure endpoint

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[shard]async fn results(cx:&Cx,query:String)->Result{if query.chars().count()>50{return
Err(bad_request().into());}let rows=search(cx,&query).await?;view!{<ul>for row in rows{<li>(row.title)</li>}
</ul>}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

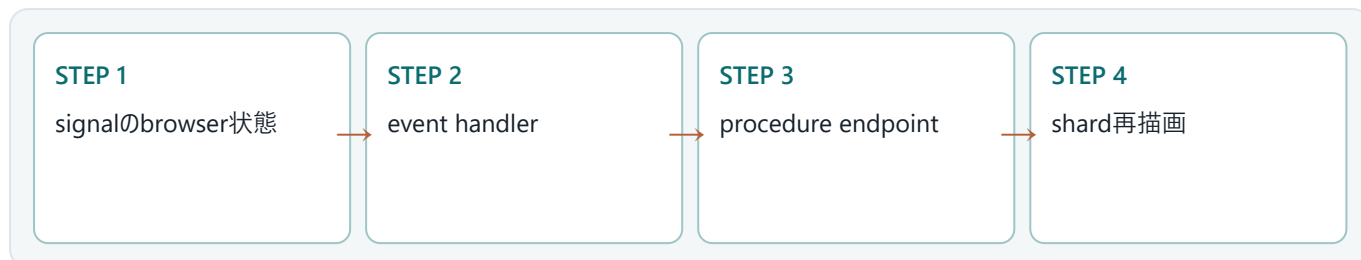
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 ・ BACKEND APPLICATION

バックエンドのどこで使うか

UI上で隠してもprocedureは直接呼べます。user_idを引数から信用せずsessionから決定し、値の範囲と対象所有者をserverで検証します。



リクエスト側

- signalのbrowser状態を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- event handlerを小さな責務へ分割
- procedure endpointをResultとログで表現
- shard再描画をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	signalのbrowser状態に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	event handlerの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	procedure endpointまたはshard再描画の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09 · DEBUGGING

runtime signal・procedure・shardを再現して切り分ける

runtime signal・procedure・shardの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: signalのbrowser状態
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: event handler
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: procedure endpoint
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: shard再描画

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	signalのbrowser状態	signalのbrowser状態 — 入力と前提を確認し、境界を曖昧にしない。
2	event handler	event handler — 値がどの順で変換されるかを追跡する。
3	procedure endpoint	procedure endpoint — 失敗を再現し、型またはログから原因を特定する。
4	shard再描画	shard再描画 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 browser反応とserver処理の境界を区別し、procedure/shardを公開APIとして守ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

検索signalをshardへ渡し、server側で最大50文字へ検証して結果を返してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[shard]async fn results(cx:&Cx,query:String)->Result{if query.chars().count(>50){return Err(bad_request().into());}let rows=search(cx,&query).await?;view!{<ul>for row in rows{<li>(row.title)</li>}</ul>}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ browser反応とserver処理の境界を区別し、procedure/shardを公開APIとして守ります。
- ✓ runtime signal・procedure・shardとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

Topcoatエラー・cookie・session・CSRF

UNIT 10 ・ CONCEPT

Topcoatエラー・cookie・session・CSRF

router error helperで400、401、403、404、redirectを表し、sessionは生tokenをcookie、hashと期限をDBに持ちます。.sessionsはorigin検証も登録します。

この単元の到達点

- RouterErrorExt
- hardened cookie
- token hash保存
- Origin検証

なぜバックエンドで必要か

認証は関数としてcurrent_userへまとめ、認可は対象SQLへowner_idを含めます。Topcoat sessionの保存先はアプリ側で実装します。



UNIT 10 ・ MENTAL MODEL

Topcoatエラー・cookie・session・CSRFを図でつかむ



router error helperで400、401、403、404、redirectを表し、sessionは生tokenをcookie、hashと期限をDBに持ちます。.sessionsはorigin検証も登録します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	RouterErrorExt	RouterErrorExt — 入力と前提を確認し、境界を曖昧にしない。
2	hardened cookie	hardened cookie — 値がどの順で変換されるかを追跡する。
3	token hash保存	token hash保存 — 失敗を再現し、型またはログから原因を特定する。
4	Origin検証	Origin検証 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Topcoatエラー・cookie・session・CSRFとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

Topcoatエラー・cookie・session・CSRFだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
let router=Router::builder().discover().cookies().sessions(SessionConfig::default()).build();
#[page("/account")]async fn account(cx:&Cx)->Result{let
user=current_user(cx).await?.ok_or_redirect("/login")?.view!{<h1>"Account of "(&user.name)</h1>}}
#[route(POST "/logout")]async fn logout(cx:&Cx)->Result<SeeOther>{if let Some(hash)=session::stop(cx).await?
{delete_session(&hash).await?;}Ok(see_other("/"))}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
let router=Router::builder().discover().cookies().sessions(SessionConfig::	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
#[page("/account")]async fn account(cx:&Cx)->Result{let user=current_user(	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
#[route(POST "/logout")]async fn logout(cx:&Cx)- >Result<SeeOther>{if let S	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力RouterErrorExtの条件を満たすか確認する。
- ✓ hardened cookieを実行し、途中の型と状態を追う。
- ✓ token hash保存で失敗を成功経路から分離する。
- ✓ Origin検証により、出力と副作用を検証する。

型を声に出す Topcoatエラー・cookie・session・CSRFに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

未認証GET /account → redirect /login
POST /logout → cookie破棄 + DB行削除 + 303

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct StoredSession{token_hash:String,user_id:i64,expires_at:i64}fn save(rows:&mut
Vec<StoredSession>,s:StoredSession){rows.retain(|x|x.token_hash!=s.token_hash);rows.push(s)}fn
revoke(rows:&mut Vec<StoredSession>,hash:&str){rows.retain(|x|x.token_hash!=hash)}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

認証は関数としてcurrent_userへまとめ、認可は対象SQLへowner_idを含めます。Topcoat sessionの保存先はアプリ側で実装します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	RouterErrorExtに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	hardened cookieの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	token hash保存またはOrigin検証の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

Topcoatエラー・cookie・session・CSRFを再現して切り分ける

Topcoatエラー・cookie・session・CSRFの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: RouterErrorExt
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: hardened cookie
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: token hash保存
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Origin検証

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	RouterErrorExt	RouterErrorExt — 入力と前提を確認し、境界を曖昧にしない。
2	hardened cookie	hardened cookie — 値がどの順で変換されるかを追跡する。
3	token hash保存	token hash保存 — 失敗を再現し、型またはログから原因を特定する。
4	Origin検証	Origin検証 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 session mechanicsはTopcoat、user/session永続化と認可はアプリが責任を持ちます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

session::startのtoken_hashとexpires_atをDBへ保存し、logoutで同じhashを削除する擬似Repositoryを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct StoredSession{token_hash:String,user_id:i64,expires_at:i64}fn save(rows:&mut Vec<StoredSession>,s:StoredSession){rows.retain(|x|x.token_hash!=s.token_hash);rows.push(s)}fn revoke(rows:&mut Vec<StoredSession>,hash:&str){rows.retain(|x|x.token_hash!=hash)}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ session mechanicsはTopcoat、user/session永続化と認可はアプリが責任を持ちます。
- ✓ Topcoatエラー・cookie・session・CSRFとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

認証付きTopcoatミニアプリを0.5.0固定で構築する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	HTTP要求・応答・状態コード	画面の裏ではHTTP契約があり、method・status・header・bodyを個別に検証します。
02	Topcoat 0.5.0最小アプリ	0.5.0のcrate、CLI、API、Cargo.lockを一組で固定します。
03	router・page・route・layout	画面はpage、機械向け応答はroute、共通骨格はlayoutに分けます。
04	path_param・query_params・Cx	0.5.0のpath parameterは属性付きtuple structとCx helperで読みます。
05	view!・component・HTML escaping	view!へ値として渡し、自分で文字列連結したHTMLを作らないことがXSS防止の基本です。
06	body extractor・form・JSON・PRG	bodyを型へparseし、副作用前に検証し、成功後は303でGETへ戻します。
07	app_context・request helper・memoize	長寿命依存はAppState、request固有計算はCx helperとmemoizeへ置きます。
08	assets・CSS・bundle	0.5.0ではrelease binaryを先にbuildし、target/assetsをbinaryと一緒に配布します。
09	runtime signal・procedure・shard	browser反応とserver処理の境界を区別し、procedure/shardを公開APIとして守ります。
10	Topcoatエラー・cookie・session・CSRF	session mechanicsはTopcoat、user/session永続化と認可はアプリが責任を持ちます。

完了条件

- ✓ 全routeのmethodとstatusが説明できる
- ✓ 入力が副作用前に検証される
- ✓ asset bundleをrelease binaryと配布する
- ✓ procedureとsessionに認証・認可テストがある

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. HTTP要求・応答・状態コードを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. Topcoat 0.5.0最小アプリを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. router・page・route・layoutを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. path_param・query_params・Cxを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. view!・component・HTML escapingを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. body extractor・form・JSON・PRGを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. app_context・request helper・memoizeを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. assets・CSS・bundleを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. runtime signal・procedure・shardを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. Topcoatエラー・cookie・session・CSRFを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. HTTP要求・応答・状態コード

HTTP要求・応答・状態コードとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 画面の裏ではHTTP契約があり、method・status・header・bodyを個別に検証します。

02. Topcoat 0.5.0最小アプリ

Topcoat 0.5.0最小アプリとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 0.5.0のcrate、CLI、API、Cargo.lockを一組で固定します。

03. router・page・route・layout

router・page・route・layoutとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 画面はpage、機械向け応答はroute、共通骨格はlayoutに分けます。

04. path_param・query_params・Cx

path_param・query_params・Cxとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 0.5.0のpath parameterは属性付きtuple structとCx helperで読めます。

05. view!・component・HTML escaping

view!・component・HTML escapingとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: view!へ値として渡し、自分で文字列連結したHTMLを作らないことがXSS防止の基本です。

06. body extractor・form・JSON・PRG

body extractor・form・JSON・PRGとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: bodyを型へparseし、副作用前に検証し、成功後は303でGETへ戻します。

07. app_context・request helper・memoize

app_context・request helper・memoizeとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 長寿命依存はAppState、request固有計算はCx helperとmemoizeへ置きます。

08. assets・CSS・bundle

assets・CSS・bundleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 0.5.0ではrelease binaryを先にbuildし、target/assetsをbinaryと一緒に配布します。

09. runtime signal・procedure・shard

runtime signal・procedure・shardとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: browser反応とserver処理の境界を区別し、procedure/shardを公開APIとして守ります。

10. Topcoatエラー・cookie・session・CSRF

Topcoatエラー・cookie・session・CSRFとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: session mechanicsはTopcoat、user/session永続化と認可はアプリが責任を持ちます。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
HTTP要求・応答・状態コード	画面の裏ではHTTP契約があり、method・status・header・bodyを個別に検証します。	TopcoatのpageはHTML、routeはAPI値を返します。ハンドラより前後のHTTP契約を理解して404、400、409、500を使い分けま す。
Topcoat 0.5.0最小アプリ	0.5.0のcrate、CLI、API、Cargo.lockを一組で固 定します。	開発はtopcoat devで再build・asset bundle・restartをまとめ ます。cargo runだけではassetを使うアプリがbundle未生成で失 敗できます。
router・page・route・layout	画面はpage、機械向け応答はroute、共通骨 格はlayoutに分けます。	URL構造をmoduleへ対応させ、共有HTMLはlayout、機械向 け応答はroute、利用者向け画面はpageへ分けます。
path_param・ query_params・Cx	0.5.0のpath parameterは属性付きtuple struct とCx helperで読みます。	IDと検索条件は未信頼入力です。型parse後も正数、上限、文 字数、認可を検証し、SQLへparameter bindします。
view!・component・HTML escaping	view!へ値として渡し、自分で文字列連結した HTMLを作らないことがXSS防止の基本です。	SSR出力は最初からHTMLとして届きます。利用者入力をraw HTMLへ変換せず、URL、属性、CSS classも文脈に応じて制約 します。
body extractor・form・ JSON・PRG	bodyを型へparseし、副作用前に検証し、成功 後は303でGETへ戻します。	検証エラーでは入力値を保持したフォームを400で再表示し、成功 時だけDBを更新してredirectします。二重送信には一意制約や idempotencyも使います。
app_context・request helper・memoize	長寿命依存はAppState、request固有計算は Cx helperとmemoizeへ置きます。	AppStateは長寿命でSend+Syncなhandleだけを持ち、リクエス ト固有の入力やロックguardを保存しません。
assets・CSS・bundle	0.5.0ではrelease binaryを先にbuildし、 target/assetsをbinaryと一緒に配布します。	binaryと同じsourceからbundleしたtarget/assetsをコンテナ の/app/assetsへコピーします。古いbundleとの不一致をCIで防ぎ ます。
runtime signal・ procedure・shard	browser反応とserver処理の境界を区別し、 procedure/shardを公開APIとして守ります。	UI上で隠してもprocedureは直接呼べます。user_idを引数から 信用せずsessionから決定し、値の範囲と対象所有者をserver で検証します。
Topcoatエラー・cookie・ session・CSRF	session mechanicsはTopcoat、user/session 永続化と認可はアプリが責任を持ちます。	認証は関数としてcurrent_userへまとめ、認可は対象SQLへ owner_idを含めます。Topcoat sessionの保存先はアプリ側で実 装します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。